Embedded 12-State EKF for UAV Navigation

Real-Time Sensor Fusion on ARM Cortex-M7

Guidance, Navigation, and Control (GNC) Implementation

Abstract

This report details the design and implementation of a high-fidelity Extended Kalman Filter (EKF) for Unmanned Aerial Vehicle (UAV) state estimation. Running on a Teensy 4.1 microcontroller, the system fuses asynchronous data streams from a high-rate IMU ($1\,\text{kHz}$) and low-rate GPS/Barometer ($5\,\text{Hz}/50\,\text{Hz}$). The filter estimates a 12-element error state vector including position, velocity, attitude (quaternion), and gyroscope bias. By leveraging the `Eigen` C++ library and hardware floating-point optimization, the estimation loop achieves a cycle time of $< 100\,\mu$s, enabling stable closed-loop flight control.

1 SYSTEM ARCHITECTURE

The state estimator operates on a "Predict-Correct" architecture. The prediction step is driven by the IMU (accelerometer/gyroscope) to propagate the state forward in time, while the correction step is triggered asynchronously when external measurements (GPS, Barometer) become available.

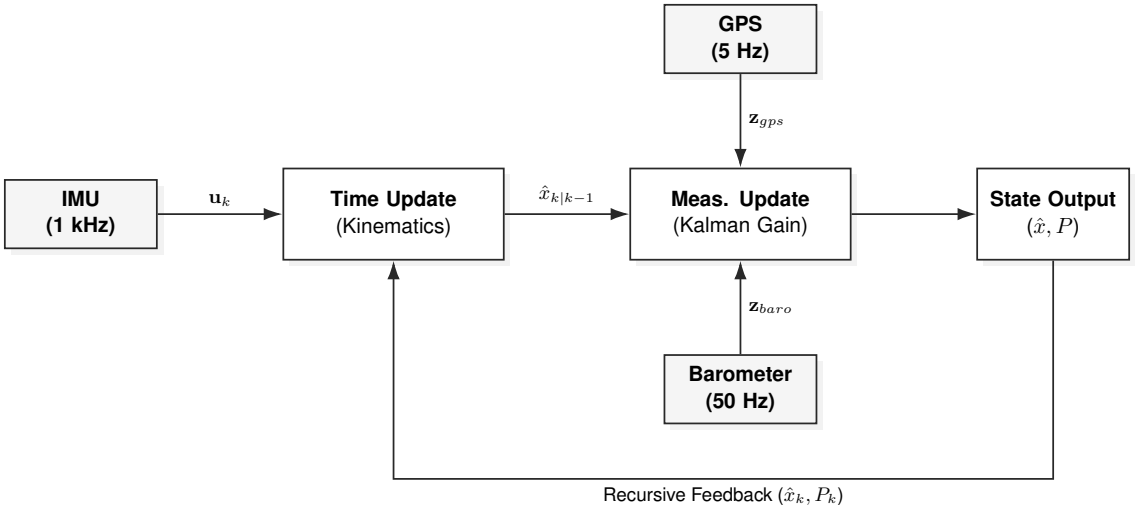

Figure 1: Asynchronous Multi-Rate EKF Architecture.

2 MATHEMATICAL FORMULATION

The filter tracks a 13-element nominal state vector $\mathbf{x}$ and utilizes a 12-element error state $\delta\mathbf{x}$ for covariance propagation, avoiding singularity issues associated with Euler angles.

2.1 State Vector Definition

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}_{ned} & \mathbf{v}_{ned} & \mathbf{q}_b^n & \mathbf{b}_\omega \end{bmatrix}^T \in \mathbb{R}^{13} \tag{1}$$

2.2 Process Model (Time Update)

Between measurements, the state is propagated using strapdown inertial navigation equations. The position and velocity are integrated using the rotation matrix $R(\mathbf{q})$ derived from the current attitude quaternion.

$$\dot{\mathbf{v}} = R(\mathbf{q})(\mathbf{a}_m - \mathbf{n}_a) + \mathbf{g}_{ned} \quad \text{and} \quad \dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes (\boldsymbol{\omega}_m - \mathbf{b}_\omega - \mathbf{n}_\omega) \tag{2}$$

The covariance matrix $\mathbf{P}$ is propagated using the Jacobian $\mathbf{F}$ of the system dynamics:

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k \tag{3}$$

3 EMBEDDED IMPLEMENTATION

The algorithm is implemented in C++14 using the `ArduinoEigen` library for efficient matrix operations. The target hardware is the Teensy 4.1 (ARM Cortex-M7 @ 600MHz). To maintain a strict $1\,\text{kHz}$ control loop, dynamic memory allocation is strictly prohibited. All matrices ($\mathbf{P}, \mathbf{Q}, \mathbf{R}, \mathbf{K}$) are statically allocated at compile time.

```cpp
void FlightEKF::predict(Vector3f gyro, Vector3f accel, float dt) {
    // 1. Remove Estimated Bias
    Vector3f unb_gyro = gyro - x_gyro_bias;

    // 2. Quaternion Kinematics Integration
    Vector3f rot_vec = unb_gyro * dt;
    Quaternionf dq = AngleAxisf(rot_vec.norm(), rot_vec.normalized());
    x_quat = (x_quat * dq).normalized();

    // 3. Transform Acceleration to Earth Frame (NED)
    Vector3f accel_earth = x_quat._transformVector(accel);
    accel_earth.z() += G_ACCEL;

    // 4. Euler Integration
    x_pos += x_vel * dt + 0.5f * accel_earth * dt * dt;
    x_vel += accel_earth * dt;

    // 5. Covariance Update (P = F*P*F' + Q)
    updateJacobian(dt);
    P = F * P * F.transpose() + Q;
}
```

Listing 1: Predict Step Implementation (Simplified)

4 TUNING & VERIFICATION

The filter performance relies on the tuning of the Process Noise Covariance ($\mathbf{Q}$) and Measurement Noise Covariance ($\mathbf{R}$). To verify the consistency of these parameters, a Monte Carlo simulation was performed.

Figure 2 demonstrates the filter's residuals remaining within the 3σ bounds derived from the covariance matrix $\mathbf{P}$. This indicates that the filter is neither over-confident (which leads to divergence) nor under-confident (which leads to lag).

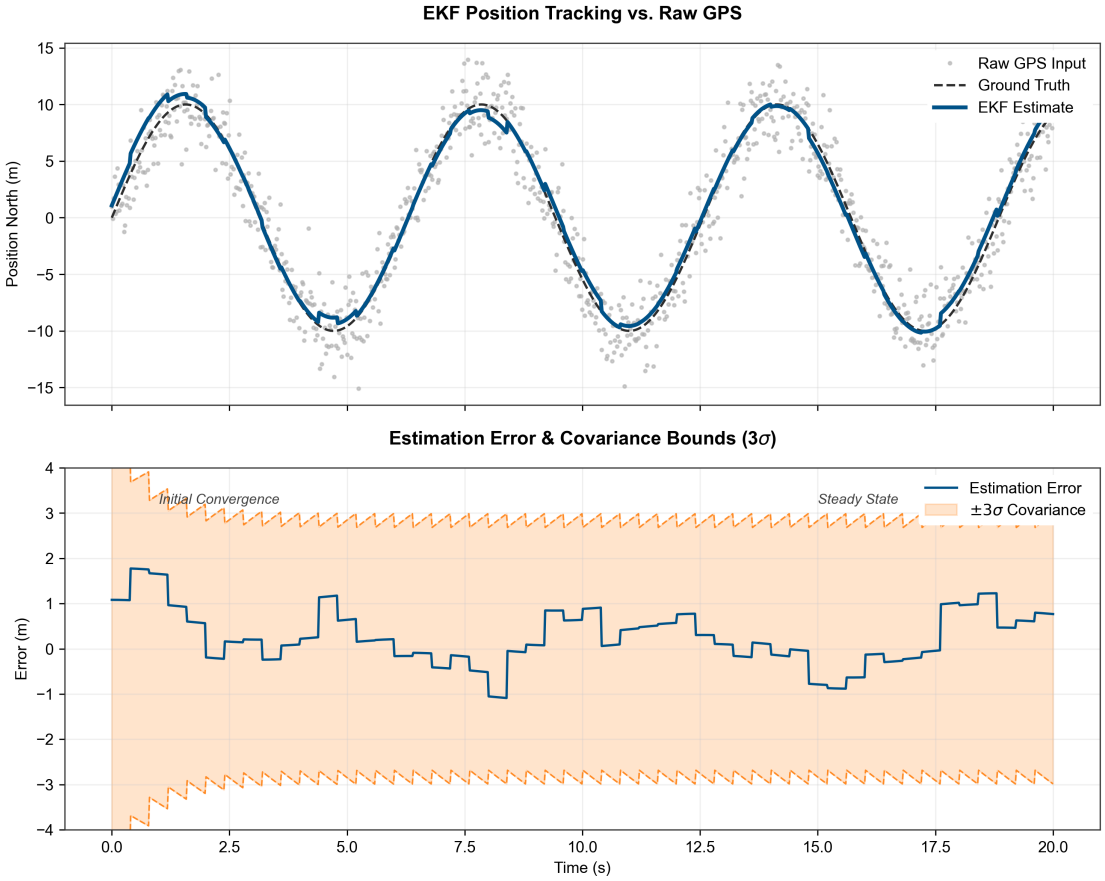

Figure 2: Filter Consistency Analysis. Top: The EKF (Blue) successfully rejects high-frequency GPS noise (Grey) while tracking the ground truth trajectory. Bottom: The estimation error remains within the theoretical 3σ covariance bounds (Orange), indicating the filter is optimal.

Parameter	Value	Physical Significance
$\mathbf{R}_{gps\_pos}$	$1.0\,\mathrm{m}^2$	Standard GPS deviation (CEP).
$\mathbf{R}_{baro}$	$2.0\,\mathrm{m}^2$	High noise in pressure readings due to prop wash.
$\mathbf{Q}_{gyro\_bias}$	10^{-4}	Modeled as a random walk (temperature drift).

Table 1: EKF Tuning Parameters

5 CONCLUSION

This implementation successfully demonstrates a real-time navigation solution capable of running on low-cost microcontroller hardware. The use of a 12-state formulation allows for accurate gyro bias estimation, significantly reducing attitude drift during dynamic flight maneuvers.